# 📘 سوالات مصاحبه فنی Backend

**جلسه 1 : Web Fundamentals & Backend Mindset**

**تفکیک‌شده برای دو سطح: Mid-Level و Senior**

**تهیه و تنظیم : علیرضا بهمنش راد - https://www.linkedin.com/in/behmaneshrad**

---

### ❓ 1. API Contract یعنی چه و چرا مهم است؟

**✅ جواب:**

API یک قرارداد بلندمدت بین Backend و Client است. وقتی API منتشر می‌شود، کلاینت‌ها به ساختار response، نام فیلدها و رفتار آن وابسته می‌شوند. تغییر بدون توجه به backward compatibility می‌تواند کل سیستم را بشکند.

---

### ❓ 2. چرا API نباید Mirror دیتابیس باشد؟

**✅ جواب:**

چون در این صورت:

- تغییر schema دیتابیس = شکستن API
- جزئیات داخلی سیستم لو می‌رود
- API به implementation قفل می‌شود

API باید یک **View پایدار** باشد، نه خروجی مستقیم tableها.

---

### ❓ 3. Fragile API یعنی چه؟

**✅ جواب:**

APIای که با تغییر کوچک مثل حذف فیلد، تغییر نوع داده یا اسم فیلد، کلاینت‌ها را می‌شکند. Fragile API معمولاً bug ندارد ولی در طول زمان دردناک می‌شود.

---

### ❓ 4. منظور از Be Conservative in What You Send چیست؟

✅ **جواب:**

یعنی:

- کمترین داده لازم را ارسال کن
- ساختار خروجی قابل گسترش باشد
- از فیلدهای فلت و غیرقابل توسعه پرهیز شود

این اصل باعث حفظ backward compatibility می‌شود.

---

### ❓ 5. آیا اضافه کردن فیلد جدید breaking change است؟

✅ **جواب:**

خیر، اگر:

- فیلد optional باشد
- کلاینت tolerant طراحی شده باشد

حذف یا تغییر معنی فیلد breaking change است، نه اضافه‌کردن.

---

### ❓ 6. Stateless Backend یعنی چه؟

✅ **جواب:**

یعنی سرور هیچ state کاربر (session) را در memory خودش نگه نمی‌دارد. تمام stateها در Redis، DB یا JWT ذخیره می‌شوند تا هر سرور بتواند هر درخواست را پاسخ دهد.

---

### ❓ 7. چرا Stateless بودن برای Load Balancer حیاتی است؟

✅ **جواب:**

چون Load Balancer درخواست بعدی کاربر را ممکن است به سرور دیگری بفرستد. اگر state روی سرور باشد، کاربر logout می‌شود یا خطا می‌گیرد.

---

### ❓ 8. Single Point of Failure چیست؟

✅ **جواب:**

هر مؤلفه‌ای که فقط یک نسخه دارد (یک سرور، یک دیتابیس، یک Redis) و با خرابی آن کل سیستم down می‌شود، SPoF است.

---

### ❓ 9. چرا Cache نیست Optimization بلکه اصل معماری است؟

✅ **جواب:**

چون بدون cache، دیتابیس زودتر از هر چیز دیگری در scale بالا از پا درمی‌آید. Cache پیش‌نیاز مقیاس‌پذیری است.

---

### ❓ 10. چه داده‌هایی را نباید Cache کرد؟

✅ **جواب:**

- پرداخت‌ها
- داده‌های بسیار حساس
- اطلاعاتی که باید لحظه‌ای دقیق باشند

مثل inventory لحظه‌ای یا balance بانکی.

---

### ❓ 11. TTL چیست و چرا مهم است؟

✅ **جواب:**

TTL زمان اعتبار داده در cache است. بدون TTL، داده قدیمی می‌ماند و سیستم رفتار ناسازگار نشان می‌دهد.

---

### ❓ 12. Scale Up و Scale Out چه تفاوتی دارند؟

✅ **جواب:**

- Scale Up: قوی‌تر کردن یک سرور (محدود و گران)
- Scale Out: اضافه‌کردن سرورهای بیشتر (پایدار و مقیاس‌پذیر)

# 🔥 سطح Senior

---

### ❓ 13. Production-First Thinking یعنی چه؟

**✅ جواب:**

یعنی قبل از نوشتن کد، به این فکر کنیم:

- اگر سرویس کند شود چه می‌شود؟
- اگر down شود چه می‌شود؟
- اگر ۱۰ برابر load بیاید چه می‌شود؟

نه فقط اینکه «الان کار می‌کند یا نه».

---

### ❓ 14. چرا Timeout حیاتی‌تر از Retry است؟

**✅ جواب:**

Timeout جلوی گیر کردن thread و resource exhaustion را می‌گیرد،

Retry بی‌حد می‌تواند باعث **Cascading Failure** شود و کل سیستم را down کند.

---

### ❓ 15. Circuit Breaker چه مشکلی را حل می‌کند؟

**✅ جواب:**

جلوگیری از cascading failure. وقتی یک dependency خراب است، circuit باز می‌شود و درخواست‌ها سریع reject می‌شوند تا سیستم اصلی زنده بماند.

---

### ❓ 16. Fail Fast چرا رفتار سالم‌تری است؟

**✅ جواب:**

چون رد کردن سریع درخواست بهتر از نگه‌داشتن threadها و crash کردن کل سیستم است. Delay بیشتر معمولاً outage بزرگ‌تر می‌سازد.

---

## ❓ 17. Eventually Consistent یعنی چه؟

✅ **جواب:**

یعنی داده ممکن است برای مدت کوتاهی ناهماهنگ باشد، اما در نهایت همگرا می‌شود. در replication و cache بسیار رایج و قابل قبول است.

---

## ❓ 18. چرا دیتابیس «آخرین خط دفاع» است؟

✅ **جواب:**

چون:

- State حیاتی را نگه می‌دارد
- Scale کردنش سخت است
- Bottleneck اصلی سیستم است

اول cache، stateless backend و load balancing، بعد DB scaling.

---

## ❓ 19. Replication چه مزیتی دارد و چه ریسکی؟

✅ **جواب:**

مزیت: پاسخ‌گویی بهتر به read

ریسک: read → lag قدیمی → eventual consistency

---

## ❓ 20. Sharding چرا آخرین راه‌حل است؟

✅ **جواب:**

چون:

- Join سخت می‌شود
- Query پیچیده می‌شود
- Debug و migration دردناک می‌شود

برای Senior، Sharding تصمیم معماری سنگین است.

---

### ❓ 21. API Versioning را چه زمانی لازم می‌دانید؟

✅ **جواب:**

وقتی تغییر:

- backward compatible نیست
- معنی قبلی داده را می‌شکند

Versioning یعنی قبول شکست قرارداد قبلی، نه راه فرار از طراحی بد.

---

### ❓ 22. Idempotency چه مشکلی را حل می‌کند؟

✅ **جواب:**

از duplicate processing جلوگیری می‌کند؛ مثلاً کاربر با چند کلیک، چند پرداخت انجام ندهد.

---

### ❓ 23. Saga Pattern چرا جایگزین PC2 است؟

✅ **جواب:**

چون PC2 در distributed systems شکننده است. Saga با compensating action اجازه می‌دهد سیستم eventually به حالت درست برگردد.

---

### ❓ 24. Senior چگونه مقیاس‌پذیری را اندازه‌گیری می‌کند؟

✅ **جواب:**

با عدد، نه احساس:

- QPS
- P95 / P99 latency
- Error rate
- Throughput